

Projet final — Clone de Vinted

1. Présentation du projet

Vous allez construire le **frontend d'un clone de Vinted** : une marketplace de vêtements et accessoires d'occasion.

L'API backend est **fournie et fonctionne en local** sur votre machine. Votre travail consiste à créer l'interface React qui consomme cette API : catalogue avec recherche et filtres, détail des articles, création d'annonces, gestion de vos annonces, et favoris.

Le projet est fourni sous forme d'un **scaffold** (repo GitHub) contenant :

- La stack technique configurée (Vite, TypeScript, Tailwind CSS, TanStack Query)
- Le routing configuré avec des **pages placeholder** à compléter
- Un service API prêt à l'emploi (authentification automatique)

2. Modalités

Aspect	Détail
Équipes	Groupes de 3 étudiants
Travail	Hors cours, sur temps personnel (~5 semaines)
Rendu	Repository GitHub fonctionnel
Soutenance	~15 min par groupe — 6 mai 2026
Ressources	Libres (IA incluse), mais tout le code doit pouvoir être expliqué en soutenance
Git	Contributions des 3 membres visibles dans l'historique, commits réguliers

3. Démarrage

Créer votre repo

1. Un membre du groupe fork le repo scaffold sur GitHub :
 - Aller sur le repo scaffold : <https://github.com/stephane-ruhlmann/esgi-vinted-clone-scaffold>
 - Cliquer sur **Fork** (en haut à droite)
 - Décocher "Copy the `main` branch only" si la case apparaît
 - Cliquer sur **Create fork**
2. Dans les **Settings** du fork : renommer le repo si souhaité (ex : `vinted-clone-groupe-X`)
3. Toujours dans **Settings** > **Collaborators** : ajouter les 2 autres membres du groupe comme collaborateurs
4. Chaque membre clone le fork :

4. Ce qui est fourni

Structure du projet

```
vinted-clone/
├── src/
│   ├── components/
│   ├── pages/
│   │   ├── CataloguePage.tsx
│   │   ├── ArticleDetailPage.tsx
│   │   ├── PublishPage.tsx
│   │   ├── MyArticlesPage.tsx
│   │   ├── FavoritesPage.tsx
│   │   └── EditArticlePage.tsx
│   ├── hooks/
│   │   └── useCurrentUserId.ts
│   ├── lib/
│   │   └── userId.ts
│   ├── services/
│   │   └── api.ts
│   ├── types/
│   │   └── article.ts
│   ├── test/
│   │   └── setup.ts
│   ├── App.tsx
│   ├── main.tsx
│   └── index.css
```

Votre espace de travail
Vide – vos composants à créer
Pages placeholder à compléter
Votre identifiant utilisateur
Génération/lecture du userId (ne pas modifier)
Service API prêt à l'emploi
Types et constantes
Setup Vitest (si vous choisissez les tests)
Layout avec navbar (ne pas modifier sauf personnalisation)
Point d'entrée, routing et providers (ne pas modifier)
Tailwind CSS

5. API — Référence

Endpoints

Articles

Méthode	Route	Description
GET	/api/articles	Liste des articles (avec filtres, voir ci-dessous)
GET	/api/articles/:id	Détail d'un article
POST	/api/articles	Créer un article
PUT	/api/articles/:id	Modifier un article (vous devez en être le propriétaire)
DELETE	/api/articles/:id	Supprimer un article (vous devez en être le propriétaire)

6. Fonctionnalités obligatoires

6.1 Catalogue (3 pts)

Objectif : afficher les articles, permettre de chercher, filtrer et trier.

Attendus :

- Afficher les articles dans une **grille de cartes** (image, titre, prix, catégorie, état, vendeur)
- Chaque carte est un **lien cliquable** vers la page de détail
- **Barre de recherche** : filtre les articles par texte (titre + description)
- **Filtre par catégorie** : menu déroulant avec les catégories + option "Toutes"
- **Filtre par état** : menu déroulant avec les conditions + option "Tous"
- **Filtre par prix** : champs min et max
- **Tri** : menu déroulant avec "Plus récent", "Prix croissant", "Prix décroissant"

7. Fonctionnalités au choix

Choisissez **au moins 2** fonctionnalités parmi les 4 suivantes. Chaque fonctionnalité supplémentaire rapporte **+1,5 pt bonus**.

7.1 Brouillon automatique (1,5 pts)

Objectif : sauvegarder automatiquement le formulaire de création dans `localStorage` , et restaurer les données si l'utilisateur quitte et revient.

Attendus :

- Quand l'utilisateur remplit le formulaire de création, les données sont sauvegardées **automatiquement** dans `localStorage` (pas de bouton "Sauvegarder")
- S'il quitte la page et revient, les champs sont **pré-remplis** avec les données du brouillon

8. Barème (20 points + bonus)

Catégorie	Points
Fonctionnel obligatoire	10
Catalogue (liste + recherche + filtres + tri)	3
Détail article	1
Création d'annonce (formulaire + validation + redirection)	2
Mes annonces + suppression avec confirmation	1,5
Favoris (toggle + page)	2
Navigation cohérente (liens entre pages, pas de liens cassés, retour catalogue)	0,5
Fonctionnalités au choix (x2 minimum)	3

9. Qualité technique (5 pts)

Critère	Points	Ce qu'on vérifie
TypeScript strict	1	Pas de <code>any</code> , aucune erreur TypeScript (<code>npx tsc --noEmit</code> passe)
Architecture propre	2	Composants bien découpés, appels API structurés (TanStack Query ou custom hooks), pas de logique dupliquée
Gestion loading/error	1	Chaque page gérant des données affiche un état de chargement et un message en cas d'erreur
Code lisible	1	Nommage clair, code formaté (<code>pnpm format</code>), pas d'erreur lint (<code>pnpm lint</code>)

Commandes utiles

10. Rendu

Comment rendre

1. **Vérifiez** que votre branche `main` contient la version finale de votre projet
2. **Envoyez l'URL** de votre repo GitHub (le fork) via MyGES avant la deadline
3. Le repo doit être **public** ou accessible par l'enseignant (ajouté en collaborateur)
4. On clonera votre repo le jour de la soutenance — ce qui est sur `main` à ce moment-là fait foi

Pas de ZIP, pas de clé USB, pas de lien Google Drive. **Uniquement le repo GitHub.**

Checklist de rendu

Avant de rendre, vérifiez chaque point :

- `[] git clone <votre-repo>` sur une machine propre

11. Soutenance

Format

- ~15 minutes par groupe
- Devant jury, sur votre machine

Déroulement

1. **Démo fonctionnelle** (~8 min) : parcours des fonctionnalités implémentées
2. **Questions techniques** (~4 min) : choix d'architecture, explications de code
3. **Questions individuelles** (~3 min) : chaque membre répond à :

Question	Ce qu'on évalue
Une partie du code qui mériterait d'être améliorée	Capacité d'auto-critique

12. Ressources

- [React — Documentation officielle](#)
- [TanStack Query — Guide](#)
- [React Router — Documentation](#)
- [Tailwind CSS — Référence](#)
- [Vitest — Documentation](#)
- [React Testing Library — Guide](#)